Documentation Technique : EJB to REST API Generator

1. Introduction

L'outil **EJB to REST API Generator** est une application web développée en Java 21 et Spring Boot 3. Son objectif principal est d'automatiser la transformation de projets Java EJB (Enterprise JavaBeans) legacy en API REST modernes basées sur Spring Boot.

L'application permet à un utilisateur de télécharger un fichier ZIP contenant le code source d'un projet EJB. L'outil analyse ensuite statiquement le code Java pour détecter les cas d'usage (UseCases), les objets de transfert de données (DTOs), et les annotations de sérialisation (JAXB). À partir de cette analyse, il génère un projet Spring Boot complet, prêt à être compilé et déployé, exposant les anciens EJB sous forme de points de terminaison REST (endpoints) supportant les formats JSON et XML.

2. Architecture de l'Application

L'application suit une architecture classique en couches (MVC) basée sur le framework Spring Boot.

2.1. Couche de Présentation (Web)

L'interface utilisateur est rendue côté serveur à l'aide du moteur de templates **Thymeleaf**.

- `GeneratorController` : Gère les requêtes HTTP (GET, POST) pour l'affichage de la page d'accueil, l'upload du fichier ZIP, le déclenchement de l'analyse, la génération du code et le téléchargement du résultat.
- `ToolExceptionHandler` : Intercepte les erreurs globales (comme le dépassement de la taille maximale d'upload) et redirige l'utilisateur vers l'interface avec un message d'erreur clair.

2.2. Couche de Service (Orchestration)

- **`GeneratorService`** : C'est le chef d'orchestre de l'application. Il gère la logique métier de haut niveau :
  - Création d'un identifiant unique (UUID) pour chaque session de travail.
  - Décompression sécurisée du fichier ZIP (protection contre les attaques *Zip Slip*).
  - Appel du parseur pour l'analyse du code.
  - Appel du moteur de génération pour créer le nouveau projet.
  - Compression du projet généré en un nouveau fichier ZIP pour le téléchargement.

2.3. Couche d'Analyse Statique (Parsing)

- **`EjbProjectParser`** : Utilise la bibliothèque **JavaParser** pour lire et comprendre le code source Java sans avoir besoin de le compiler.
  - Il parcourt l'arborescence des fichiers extraits.
  - Il détecte les classes annotées avec `@Stateless` ou implémentant l'interface `BaseUseCase`.
  - Il identifie les méthodes `execute` et déduit les DTOs d'entrée et de sortie.
  - Il recherche activement les annotations JAXB (`@XmlRootElement` , `@XmlElement` , etc.) pour déterminer si le support XML est requis.

2.4. Couche de Génération de Code

- **`CodeGenerationEngine`** : Responsable de la création physique des fichiers du nouveau projet Spring Boot.
  - Génère le fichier `pom.xml` avec les dépendances nécessaires (incluant JAXB et Jackson XML si détectés).
  - Génère la classe principale `Application.java` et le fichier `application.properties`.
  - Génère les **Controllers REST** (`@RestController`) qui exposent les endpoints.

- Génère les **Service Adapters** qui effectuent les appels JNDI vers les EJB distants.
- Recrée les **DTOs** en conservant les annotations JAXB pour garantir la compatibilité XML.
- Génère des classes utilitaires : gestion globale des exceptions (`GlobalExceptionHandler`), aspect de journalisation (`LoggingAspect`), et configuration de négociation de contenu (`ContentNegotiationConfig`).

3. Modèles de Données

L'analyse du code source produit un modèle de données interne structuré :

- **`ProjectAnalysisResult`** : Contient le bilan global de l'analyse (nombre de fichiers, liste des UseCases, liste des DTOs, avertissements).
- **`UseCaseInfo`** : Représente un EJB détecté. Contient son nom, ses DTOs associés, le nom du endpoint REST généré, et le format de sérialisation supporté (JSON, XML, ou les deux).
- **`DtoInfo`** : Représente un objet de transfert. Contient la liste de ses champs et un indicateur de présence d'annotations JAXB.

4. Flux de Traitement (Workflow)

1. **Upload** : L'utilisateur soumet un fichier `.zip`. Le `GeneratorService` le décompresse dans un répertoire temporaire défini par `app.upload.dir`.
2. **Scan** : L'utilisateur clique sur "Scanner". Le `EjbProjectParser` lit les fichiers `.java`, remplit le modèle `ProjectAnalysisResult`, et l'interface affiche les statistiques.
3. **Génération** : L'utilisateur clique sur "Générer". Le `CodeGenerationEngine` crée l'arborescence du nouveau projet dans `app.output.dir`.
4. **Téléchargement** : L'utilisateur télécharge le résultat. Le `GeneratorService` zippe le dossier généré et le renvoie via le `GeneratorController`.

Guide d’Installation et d’Utilisation (Pas à Pas)

Ce guide est conçu pour vous accompagner pas à pas dans l’installation et le démarrage de l’application, même si vous n’avez aucune expérience préalable en développement. Suivez simplement chaque étape attentivement.

Étape 1 : Vérifier les prérequis sur votre ordinateur

Avant de commencer, votre ordinateur a besoin de deux outils essentiels pour faire fonctionner l’application : **Java** (le langage de programmation) et **Maven** (l’outil qui assemble l’application).

1.1. Vérifier si Java est installé

1. Ouvrez l’**Invite de commandes** (sur Windows, tapez `cmd` dans la barre de recherche du menu Démarrer et appuyez sur Entrée. Sur Mac, ouvrez l’application **Terminal**).
2. Dans la fenêtre noire qui s’ouvre, tapez exactement la commande suivante et appuyez sur Entrée : `java -version`
3. **Résultat attendu** : Le système doit afficher un texte indiquant la version de Java. **Attention : la version doit être 21 ou supérieure.** (Exemple : `openjdk version "21.0.2"`).
4. **Si Java n’est pas installé ou si la version est trop ancienne** : Vous devez télécharger et installer Java 21 (OpenJDK 21) depuis internet, puis redémarrer votre ordinateur.

1.2. Vérifier si Maven est installé

1. Toujours dans la même fenêtre noire (Invite de commandes ou Terminal), tapez la commande suivante et appuyez sur Entrée : `mvn -version`
2. **Résultat attendu** : Le système doit afficher des informations sur Apache Maven.

3. **Si Maven n'est pas reconnu** : Vous devez télécharger et installer Apache Maven, puis l'ajouter aux variables d'environnement de votre système.

Étape 2 : Récupérer le code de l'application

Vous devez maintenant placer les fichiers de l'application sur votre ordinateur.

1. Téléchargez le fichier ZIP contenant le code source de l'application (le fichier s'appelle généralement `ejb-to-rest-tool-complete.zip`).
2. Créez un nouveau dossier sur votre bureau (par exemple, nommez-le `GenerateurAPI`).
3. Déplacez le fichier ZIP téléchargé dans ce nouveau dossier.
4. Faites un clic droit sur le fichier ZIP et choisissez **"Extraire ici"** (ou "Extraire tout").
5. Vous devriez maintenant voir un dossier nommé `ejb-to-rest-tool` contenant plusieurs fichiers et dossiers (comme `src`, `pom.xml`, etc.).

Étape 3 : Préparer (compiler) l'application

Nous allons maintenant demander à Maven de préparer l'application pour qu'elle soit prête à être lancée.

1. Ouvrez à nouveau votre **Invite de commandes** (ou Terminal).
2. Vous devez "naviguer" vers le dossier où vous avez extrait les fichiers. Pour cela, utilisez la commande `cd` (qui signifie "change directory"). Par exemple, si votre dossier est sur le bureau, tapez : `cd Desktop/GenerateurAPI/ejb-to-rest-tool` *(Appuyez sur Entrée. Le chemin exact dépend de l'endroit où vous avez mis le dossier).*
3. Une fois dans le bon dossier, tapez la commande magique suivante pour assembler l'application : `mvn clean package -DskipTests`
4. Appuyez sur Entrée et patientez. Vous allez voir beaucoup de texte défiler à l'écran. C'est normal, Maven télécharge les éléments nécessaires.
5. **Résultat attendu** : À la fin, vous devez voir un grand message vert indiquant **"BUILD SUCCESS"**. Si vous voyez "BUILD FAILURE", vérifiez que vous êtes bien

dans le bon dossier et que Java 21 est bien installé.

Étape 4 : Lancer l'application

L'application est prête ! Il ne reste plus qu'à l'allumer.

1. Toujours dans la même fenêtre noire, tapez la commande suivante : `java -jar target/ejb-to-rest-generator-1.0.0.jar`
2. Appuyez sur Entrée. Vous allez voir le logo "Spring" apparaître en texte, suivi de plusieurs lignes d'informations.
3. L'application est démarrée lorsque vous voyez une ligne indiquant : `Started EjbToRestGeneratorApplication in ... seconds`.
4. **Important** : Ne fermez pas cette fenêtre noire ! Si vous la fermez, l'application s'arrêtera. Réduisez-la simplement.

Étape 5 : Utiliser l'application dans votre navigateur

Maintenant que le "moteur" tourne en arrière-plan, vous pouvez utiliser l'interface visuelle.

1. Ouvrez votre navigateur internet habituel (Google Chrome, Firefox, Edge, Safari…).
2. Dans la barre d'adresse en haut (là où vous tapez habituellement www.google.com), tapez exactement ceci : `http://localhost:8080`
3. Appuyez sur Entrée.
4. **Félicitations !** Vous devriez voir la page d'accueil de l'outil "EJB to REST API Generator".

Comment utiliser l'outil :

- **Étape 1 (Upload)** : Cliquez sur la zone pointillée pour sélectionner le fichier ZIP de votre ancien projet EJB, puis cliquez sur le bouton bleu "Uploader le projet".
- **Étape 2 (Scan)** : Une fois le projet uploadé, le bouton gris "Scanner le projet EJB" devient cliquable. Cliquez dessus. L'outil va analyser votre code et afficher un

tableau avec ce qu'il a trouvé (les UseCases et les formats JSON/XML).

- **Étape 3 (Générer)** : Cliquez sur le bouton vert "Générer le projet API REST". L'outil fabrique le nouveau code.
- **Étape 4 (Télécharger)** : Cliquez sur le bouton "Télécharger le projet API généré". Vous obtiendrez un nouveau fichier ZIP contenant votre API toute neuve, prête à l'emploi !

Étape 6 : Arrêter l'application

Quand vous avez terminé de travailler :

1. Retournez sur la fenêtre noire (Invite de commandes ou Terminal) que vous aviez réduite.
2. Appuyez simultanément sur les touches **Ctrl** et **C** de votre clavier.
3. L'application va s'arrêter proprement. Vous pouvez maintenant fermer la fenêtre.

Fin de la documentation.